Flow Core System

Audit de marché du coût de développement

21 octobre 2026 | Java, Rust et .NET | Audit sans modification du classeur

Conclusion

Le chiffrage actuel est explicable comme une construction d'hypothèses, mais il n'est pas encore calibré par des réalisations comparables. Les recherches permettent de remettre en cause plusieurs choix de composants et limites de périmètre. Elles ne permettent pas de certifier un nouveau total en JH.

Référence actuelle après OSS et IA	Java	Rust	.NET
Développement pur du Core	10,135 M€	12,262 M€	10,769 M€
Charge	13 513 JH	16 349 JH	14 359 JH

1. Réemploi métier à instruire. OFBiz et Moqui/Mantle proposent déjà des modèles et services de commandes et de stock. Le chiffrage actuel crédite surtout des frameworks ; F03 ne porte aucun gain OSS métier distinct.

2. Configuration à redécouper en priorité. F11 représente 2 032 JH Java. Superposition couvre une partie de la configuration contextuelle ; les cycles de définition d'Elsa et Flowable évitent aussi du code. La provenance des décisions et les effets de publication restent à traiter.

3. Périmètre à réaligner. La cartographie publiée r87 enrichit notamment les décisions de transport et les responsabilités des services financiers. Une baisse générale de toutes les lignes serait injustifiée.

4. Écarts de stacks à reconstruire. Java dispose d'atouts de réemploi concrets. Les suppléments .NET et Rust ne peuvent cependant pas être déduits de l'âge des écosystèmes ; des alternatives fonctionnelles crédibles sont disponibles.

Priorité économique : F03 + F05 + F11 concentrent 5 408 JH, soit 4,056 M€ et 40,0 % du coût Java. Ce montant est le périmètre à examiner en premier, pas une économie annoncée.

Les pages suivantes distinguent : faits vérifiés, limites de comparaison, jugement d'audit et conditions d'un futur rechiffrage. Les liens entre crochets ouvrent les sources. Les JH présentés sont ceux du classeur, pas une nouvelle estimation.

1. Base auditée et niveau de preuve

Le build étudié comprend architecture, conception, code, assemblage des composants du produit et tests génériques. Les règles, données, connecteurs et recettes propres à Beaumanoir restent dans l'Intégration business. Le pilotage demeure une enveloppe globale séparée.

Étape du calcul actuel	Java	Rust	.NET
Référence reconstituée commune	22 580	22 580	22 580
Travaux évités par OSS	-6 160	-5 690	-5 590
Assemblage additionnel	+420	+3 590	+950
Avant gain IA	16 840	20 480	17 940
Gain IA net	-3 327	-4 131	-3 581
Après OSS et IA	13 513	16 349	14 359

Point méthodologique : les 22 580 JH ont été reconstitués à partir du résiduel Java et des gains supposés. Ils ne constituent pas une estimation indépendante « sans OSS ». La cohérence arithmétique ne démontre donc pas le niveau absolu des charges.

À TJM commun de 750 €, l'écart .NET est de 846 JH : 570 JH de réemploi en moins + 530 JH d'assemblage en plus - 254 JH de gain IA supplémentaire. L'écart Rust est de 2 836 JH : 470 + 3 170 - 804. Le différentiel Rust vient surtout de l'assemblage supposé, pas d'un déficit massif de réemploi comptabilisé.

Preuve	Ce qu'elle peut justifier	Ce qu'elle ne justifie pas
Documentation / dépôt	Couverture d'une fonction, licence et API	Nombre de JH économisés
Publication algorithmique	Difficulté d'un problème et choix d'algorithme	Multiplicateur de développement
Retour de réalisation	Effort comparable si périmètre et équipe connus	Conversion automatique mois → JH
Abaque calibré	Fourchette sur taille et cohortes comparables	Ratio universel Java / Rust / .NET

L'audit n'identifie aucun abaque public qui valide les 14 montants. Il identifie en revanche des comparables fonctionnels et des références de méthode pour rendre un rechiffrage défendable. [R05] [R06]

2. Le périmètre à comparer au marché

Cartographie utilisée : **publication 2026-10-21.3, révision 87**. Les champs du backlog examinés sur ce périmètre concordent avec la publication. Les capacités servent à définir le produit générique ; les occurrences et choix propres à Beaumanoir n'entrent pas dans son build.

Bloc actuel	Lecture de la cartographie	Évolution proposée
F01	Operational References : six familles, pas seulement produits	Renommer et séparer les modèles locaux du PIM d'entreprise
F03	Stocktaking, registres, positions, propriétés et réservations	Garder la capacité générique de comptage ; déléguer les opérations propres aux exécutants
F06	13 capacités : workflows, supervision, travail humain, intelligence de processus	Préciser les indicateurs requis ; ne pas présumer un moteur complet de process mining
F07	11 capacités, dont consolidation/routage de transport et décisions de disposition	Rendre visibles les décisions ; affecter leurs modèles d'optimisation une seule fois
F08	17 capacités : biens, transport, douanes et services financiers	Séparer objets/cycles métier, résultats et documents ; clarifier facturation et paiement
F09 + F10	18 capacités : plans, stocks cibles, carnet, préparation demande/ressources	Séparer qualification des prévisions, couverture, scénarios, affectations et libération
F11	Ensemble transverse, sans domaine métier canonique unique	Distinguer contexte, cycle de publication, provenance et console

La capacité **Demand Forecast Qualification** désigne une qualification des prévisions. Elle ne justifie pas à elle seule l'ajout d'un moteur de prévision statistique. À l'inverse, la consommation commandes/prévisions et la réconciliation doivent être explicites.

Les hotspots désignent des questions à résoudre. Leur présence ne constitue ni un volume de code, ni un coefficient de complexité, ni la preuve qu'une fonction entière doit être réécrite.

Découpage recommandé : conserver une synthèse de 14 lignes et détailler seulement 3 à 5 unités d'œuvre par fonction. La proposition n'impose pas une multiplication d'onglets ou un tableau de centaines de lignes.

3. Revue des fonctions | 01-02

JH actuels après OSS et IA. Les orientations sont des jugements d’audit conditionnels.

F01 | Références opérationnelles

Java 399 JH | Rust 508 JH | .NET 420 JH

Comparables et réemploi : Akeneo CE est un comparable PIM partiel. OFBiz et Mantle sont plus proches d’un ensemble produits, parties, accords et services. Ils offrent des modèles métier, au-delà des imports Spring Batch déjà crédités.

Reste à construire / limite : FLOW tient des vues locales Supply : identité, validité, fraîcheur, provenance et incohérences. Il ne reprend pas la maîtrise d’entreprise d’un PIM/MDM. Mantle est lié à Moqui/Java/Groovy ; son modèle n’est pas une bibliothèque JPA directement interchangeable.

Ce que je changerais : Renommer « Référentiels opérationnels multi-domaines ». Distinguer socle commun, familles de références, ingestion/publication et administration/qualification. Comparer l’adaptation des schémas existants à leur redéveloppement.

Effet sur le chiffrage : Baisse possible sur conception et code des modèles, après coût d’adaptation. Les 399 JH restent non calibrés. Akeneo Reference Entities est Enterprise ; Pimcore courant ne doit pas être supposé gratuit en production à cette échelle. Une API commune réduit le différentiel de langage.

[RP01] [RP02] [RP03] [SO01] [SO03]

F02 | Politiques Supply

Java 354 JH | Rust 453 JH | .NET 420 JH

Comparables et réemploi : Flowable DMN fournit l’exécution des décisions ; Apache KIE/Drools fournit aussi des analyses de tables. RulesEngine est une option .NET pour expressions. SAP Supply Assignment et Microsoft DOM illustrent des politiques paramétrables de protection, priorité et libération.

Reste à construire / limite : Les six familles de politiques génériques, leurs données d’entrée et leurs explications restent à définir. Une table DMN et une expression JSON ne sont pas fonctionnellement identiques. La validation générale de règles arbitraires dépasse l’analyse de tables disponible.

Ce que je changerais : Séparer contrats des politiques, adaptation/validation du runtime, évaluation explicable et tests d’équivalence. Porter la publication en F11 seulement. Retenir un moteur de référence sans cumuler les économies de deux solutions substituables.

Effet sur le chiffrage : Challenger les analyses développées sur mesure et le supplément .NET. Si DMN est requis, comparer le même service pour les trois stacks. Aucun abaque ne démontre ni ne réfute les 354 JH.

[P01] [P06] [P17] [P27] [P28]

3. Revue des fonctions | 03-04

JH actuels après OSS et IA. Les orientations sont des jugements d’audit conditionnels.

F03 | Stock et réservations

Java 1 836 JH | Rust 2 095 JH | .NET 1 836 JH

Comparables et réemploi : Mantle expose disponibilité, réservations et détails de mouvements ; Odoo Community stock porte quantités, propriétaire, lots, réservations et inventaires. ERPNext documente les corrections de ledger et réservations. Le modèle actuel ne crédite aucun réemploi métier OSS distinct sur cette ligne.

Reste à construire / limite : Ces produits ne démontrent pas le rapprochement FLOW entre photographies et événements, les garanties de concurrence, ni le cycle complet du transfert de propriété. Un champ propriétaire ou une contre-passation ne couvre pas tout le produit. Leur architecture et leur licence doivent être prises en compte.

Ce que je changerais : Décomposer registre/positions, réservations/protections, vérification/correction, propriété et restitutions. Tester un retard, un doublon, une correction tardive et deux réservations concurrentes sur un candidat avant d’estimer le reste.

Effet sur le chiffrage : Priorité de baisse à challenger sur le socle standard. Les 1 836 JH ne peuvent être remplacés par un pourcentage arbitraire. F03 conserve les invariants métier ; F12 fournit le transport fiable ; F10 décide l’affectation. Les interfaces Beaumanoir restent hors Core.

[SO03] [SO05] [SO06] [SO07] [SO08] [SO17]

F04 | Disponibilité et promesse

Java 1 000 JH | Rust 1 138 JH | .NET 1 000 JH

Comparables et réemploi : Microsoft documente séparément ATP projeté et CTP avec capacité. Mantle et les modules d’inventaire couvrent une partie de la disponibilité. IBM distingue des objectifs déterministes de coût/capacité et des enrichissements prédictifs.

Reste à construire / limite : Une disponibilité arithmétique, une décision de capacité sous contraintes et un arbitrage économique n’ont pas la même difficulté. Le cycle proposition/validité/réexamen/engagement et sa concurrence restent nécessaires, même si le calcul ATP est simple.

Ce que je changerais : Créer cinq sous-ensembles : ATP courant, ATP projeté, CTP, PTP et cycle de promesse. Préciser horizons, volumes, délais de réponse et garantie indicative ou ferme. Utiliser les projections F03, politiques F02, solveur F09 et engagements F05.

Effet sur le chiffrage : Baisse possible de l’ATP par réemploi et partage. CTP/PTP restent à dimensionner ; ne pas ajouter automatiquement du ML, de la prévision ou un ordonnancement industriel complet. Les 1 000 JH globaux masquent cette différence de portée.

[SO05] [SO18] [SO20]

3. Revue des fonctions | 05-06

JH actuels après OSS et IA. Les orientations sont des jugements d’audit conditionnels.

F05 | Commandes Fashion

Java 1 540 JH | Rust 1 779 JH | .NET 1 540 JH

Comparables et réemploi : OFBiz et Mantle couvrent ventes, achats et retours ; le code de test Mantle montre un parcours commande/réservation. Medusa documente modifications, retours, échanges et réclamations. Le crédit actuel de 150 JH ne valorise que persistance et validation.

Reste à construire / limite : La cartographie D04 compte huit capacités et 24 comportements : sept types de commandes et leur structuration. Ces capacités partagent un noyau commun. Consignations dans les deux sens, conservation lors des scissions/fusions et engagements doivent être confrontés au code existant.

Ce que je changerais : Distinguer noyau commun, ventes/achats/transferts, retours/résolutions, consignation et structuration. Classer chaque comportement comme couvert, configurable, extension ou nouveau développement. Garder les décisions de disposition en F07 et le paiement effectif dans le périmètre financier clarifié.

Effet sur le chiffrage : Candidate forte à une baisse par réemploi métier ; ni le nombre de services Mantle ni celui de modules ne donne un gain en JH. Le choix d’un socle métier et son adaptation peuvent peser davantage que le seul langage du Core.

[SO01] [SO03] [SO04] [SO12] [SO14]

F06 | Processus et tâches

Java 709 JH | Rust 1 085 JH | .NET 997 JH

Comparables et réemploi : Flowable fournit BPMN/CMMN/DMN. Elsa propose déjà définitions versionnées, publication, import/export, API et studio. Le SDK Rust de Temporal est GA depuis septembre 2026 et ouvre une option d’orchestration durable en code.

Reste à construire / limite : Exécuteur, tâches humaines, concepteur, supervision et indicateurs sont des objets distincts. Les anciennes UI et moteurs Form/Content ont quitté Flowable OSS 7. Temporal n’est pas une suite BPMN/CMMN équivalente ; Elsa doit être évalué sur le travail humain demandé.

Ce que je changerais : Séparer exécution/reprise, tâches et affectations, supervision/intervention, indicateurs et cycle des définitions partagé avec F11. Vérifier édition, écrans et droits effectivement réemployés. Les cas Flowable de quelques jours ou mois portent sur des applications construites sur une plateforme existante.

Effet sur le chiffrage : Réévaluation dans les deux sens : réemploi accru possible, mais UI manquante à financer si nécessaire. Les suppléments de 288 JH .NET et 376 JH Rust doivent correspondre à des lacunes documentées, pas à une préférence de stack.

[P01] [P02] [P04] [P05] [P11] [P19]

3. Revue des fonctions | 07-08

JH actuels après OSS et IA. Les orientations sont des jugements d’audit conditionnels.

F07 | Orchestration fulfilment

Java 736 JH | Rust 905 JH | .NET 781 JH

Comparables et réemploi : Medusa, OneStock, Fluent et IBM donnent des comparables de sélection et orchestration fulfilment. OR-Tools fournit des modèles de routage. Les cas Monsoon et Psycho Bunny décrivent des déploiements OMS partiels, pas la construction d’un Core.

Reste à construire / limite : La cartographie D06 inclut désormais plans de transport, consolidation, routage/horaires, visibilité opérationnelle et décisions de disposition des biens. Ces fonctions dépassent une simple sélection de point de préparation. Leur couverture n’est pas visible dans l’enveloppe actuelle.

Ce que je changerais : Afficher capacités/sélection, proposition/engagement, coordination/preuve, transport et disposition/libération. Affecter les modèles métier à leur fonction et le solveur partagé une seule fois en F09. Séparer libération de prestation et libération commerciale F10.

Effet sur le chiffrage : Ne pas réduire les 736 JH avant réconciliation du périmètre. Un complément est possible si transport/disposition sont absents ailleurs ; réemploi et mutualisation peuvent le limiter. Aucune réécriture de WMS/TMS ni protocole C-LOG spécifique n’est ajouté au Core.

[SO13] [SO20] [SO24] [SO26] [SO29]

F08 | Services et documents

Java 630 JH | Rust 749 JH | .NET 658 JH

Comparables et réemploi : FreeMarker, Scriban et Tera évitent un interpréteur de templates ; Gotenberg fournit un rendu via API commune. OFBiz/Mantle apportent des modèles transactionnels utiles. La ligne porte toutefois 17 capacités métier de Service Management.

Reste à construire / limite : Le chiffrage parle de demandes financières et exclut facturation/encaissement ; la cartographie décrit Billing et Payment avec corrections, paiements partiels, refus et remboursements. Il faut préciser ce que FLOW réalise ou délègue, sans importer fiscalité ou connecteurs Beaumanoir.

Ce que je changerais : Séparer contrat commun, familles logistiques, Billing/Payment génériques et justificatifs. Réutiliser cycles F05/F06 ; chiffrer uniquement les extensions. Porter explicitement états, résultats et délégation des services financiers avant de déclarer la couverture complète.

Effet sur le chiffrage : Les 630 JH ne sont pas validables sur le seul argument PDF. Les écarts actuels de 28 JH .NET et 119 JH Rust ne découlent pas des moteurs de templates. Réduire leur part documentaire si le même service de rendu est retenu, puis revoir le reste métier.

[RP10] [RP12] [RP13] [RP14] [SO03]

3. Revue des fonctions | 09-10

JH actuels après OSS et IA. Les orientations sont des jugements d’audit conditionnels.

F09 | Master Plan et optimisation

Java 1 429 JH | Rust 1 637 JH | .NET 1 429 JH

Comparables et réemploi : OR-Tools propose le même moteur via interfaces officielles Java et C#. Timefold enrichit l’option Java ; frePPLe est un candidat plus intégré de planification. Le cas RELEX/Takko couvre taille/couleur, saisonnalité et redistribution, avec un périmètre plus large que FLOW.

Reste à construire / limite : Le solveur ne fournit pas le modèle Fashion, les objectifs, les contraintes, les scénarios ni leur application. Les travaux scientifiques sur lot-sizing et allocation Fashion montrent une difficulté dépendante des contraintes ; certains problèmes simplifiés ont des solutions plus accessibles.

Ce que je changerais : Séparer préparation/netting, politiques de stock, modèles d’optimisation, scénarios/explications et application. Inclure qualification et consommation des prévisions, sans ajouter un moteur statistique. Comparer OR-Tools seul à un service de planification existant. Borner taille, temps de résolution et qualité acceptable.

Effet sur le chiffrage : Ligne à forte incertitude : baisse par modèles/UI réemployés ou complément pour exigences omises. Les 1 250 JH évités avant IA sont un contre-factuel local, pas un abaque Google. La parité Java/.NET sur le moteur est étayée ; les 208 JH Rust supplémentaires restent à décomposer.

[RP16] [RP17] [RP18] [RP19] [RP20] [RP22] [RP25] [RP26] [RP27]

F10 | Carnet et affectations

Java 777 JH | Rust 851 JH | .NET 777 JH

Comparables et réemploi : SAP Supply Assignment est un comparable direct d’affectation en pénurie, contrôle de libération et désaffectation. Les recherches Fashion illustrent des allocations contraintes. Elles ne permettent pas de traiter toute affectation comme un simple tri FIFO.

Reste à construire / limite : Priorisation simple, contraintes couplées et réaffectation conservant les engagements doivent être séparées. La décision de réserver ou libérer appartient à F10 ; la matérialisation des quantités à F03. Les paramètres réels par marque restent hors Core.

Ce que je changerais : Décomposer carnet/critères, affectation déterministe, affectation contrainte/réaffectation, politique de réservation/libération. Mutualiser moteur F09 et règles F02 ; préciser l’interface de décision et les explications sans créer un autre solveur.

Effet sur le chiffrage : Challenger à la baisse le cockpit et les mécanismes simples réutilisables. Dimensionner séparément les contraintes Fashion et la stabilité des engagements. Aucun comparable trouvé ne valide les 777 JH ni une surcharge particulière de langage sur toute la fonction.

[P28] [SO15] [SO27] [SO28]

3. Revue des fonctions | 11-12

JH actuels après OSS et IA. Les orientations sont des jugements d'audit conditionnels.

F11 | Configuration gouvernée

Java 2 032 JH | Rust 2 523 JH | .NET 2 278 JH

Comparables et réemploi : Superposition, écrit en Rust et distribué sous Apache 2.0, fournit configuration contextuelle, dimensions, surcharges, validation, versions, audit et UI/API. Son adoption par Hyperswitch est documentée. Elsa et Flowable couvrent aussi des cycles de définitions.

Reste à construire / limite : Aucune preuve de couverture intégrale de l'approbation de toutes les configurations, de la publication coordonnée planifiée, de la simulation Supply ou de HS-023. Revenir à une version de configuration n'annule pas les effets métier déjà produits. La provenance décisionnelle relie entrées, versions, décisions et effets.

Ce que je changerais : Séparer configuration contextuelle, gouvernance/publication, provenance décisionnelle et console/simulation. Évaluer Superposition pour les trois stacks et Git/CI pour les artefacts techniques lorsque les utilisateurs visés le permettent.

Effet sur le chiffrage : Première priorité de redimensionnement : 2 032 JH Java. Les +246 JH .NET et +491 JH Rust sont insuffisamment étayés ; une publication interservices est un choix d'architecture. Réduction candidate sur les mécanismes déjà couverts, sans réduire arbitrairement la provenance ni la qualification.

[P07] [P08] [P09] [P10] [P04] [P25] [P26]

F12 | API et événements

Java 516 JH | Rust 698 JH | .NET 571 JH

Comparables et réemploi : Spring Modulith offre registre transactionnel de publication et resoumission ; Wolverine apporte inbox/outbox durable en .NET. Camel couvre des patterns et connecteurs, utiles seulement là où le produit générique en a effectivement besoin.

Reste à construire / limite : Les composants n'écrivent pas les contrats canoniques, la compatibilité de versions ni les garanties d'effets métier. Un catalogue de connecteurs n'est pas un gain sur des interfaces Beaumanoir exclues. Une architecture entièrement distribuée n'est pas une exigence acquise.

Ce que je changerais : Distinguer contrats, livraison fiable, rejeu/erreurs et SDK/documentation/simulateurs. Évaluer d'abord les frontières qui nécessitent réellement réseau, broker et coordination. Compter une seule fois retries, sécurité HTTP et observabilité.

Effet sur le chiffrage : Baisse possible de mécanismes réécrits ou de distribution inutile. Les +55 JH .NET et +182 JH Rust exigent des tâches identifiées. MassTransit v9 ne doit pas remplacer silencieusement Wolverine dans une hypothèse OSS gratuite.

[P13] [P14] [P18]

3. Revue des fonctions | 13-14

JH actuels après OSS et IA. Les orientations sont des jugements d'audit conditionnels.

F13 | Socle technique

Java 611 JH | Rust 829 JH | .NET 656 JH

Comparables et réemploi : Spring fournit un socle Java riche. ASP.NET Core et ABP montrent une offre .NET modulaire couvrant aussi infrastructure et conventions. PostgreSQL, OIDC/Keycloak et une partie de l'outillage peuvent être communs aux scénarios.

Reste à construire / limite : ABP distingue framework libre et modules PRO ; la présence d'un module commercial ne donne pas un crédit OSS. Les autorisations propres au produit, migrations, packaging et diagnostics doivent rester cohérents avec l'architecture choisie.

Ce que je changerais : Décomposer droits produit/OIDC, packaging/migrations, instrumentation et chaîne de construction/tests. Examiner le résiduel après les gains déjà déduits ; ne pas retrancher une seconde fois les composants communs inclus dans la référence.

Effet sur le chiffrage : La proximité Java/.NET est crédible sur ce socle. Les 611 JH Java restent à justifier par unités ; les +218 JH Rust doivent découler de briques manquantes ou d'assemblage démontrés. Aucune marge générale fondée seulement sur l'âge du langage.

[P13] [P15] [P16] [P30]

F14 | Qualification produit

Java 944 JH | Rust 1 099 JH | .NET 996 JH

Comparables et réemploi : Testcontainers propose des dépendances jetables pour Java, .NET et Rust. OWASP ASVS fournit des exigences de sécurité utilisables pour borner les vérifications. Ces outils et standards structurent la qualification mais ne donnent pas sa charge.

Reste à construire / limite : Les oracles métier, invariants interdomaines, profils de charge, scénarios de panne et compatibilités de version restent propres au produit. Ils ne sont ni les tests locaux déjà chiffrés par fonction, ni la recette ou les données Beaumanoir.

Ce que je changerais : Construire une matrice exigence × scénario × volume × résultat attendu. Identifier campagnes automatisées, correction transverse et preuves de reprise/sécurité. Affecter outillage réutilisable à F13 et essais locaux aux fonctions concernées.

Effet sur le chiffrage : Les 944 JH représentent une provision insuffisamment étayée, pas un montant à supprimer. Réestimer sur la couverture attendue et les risques d'architecture ; aucun abaque public trouvé ne permet une réduction forfaitaire ni les suppléments de stack actuels.

[P30] [P31] [R05]

4. Comparer des assemblages complets

Le langage du Core et celui d'un composant réemployé ne sont pas obligatoirement identiques. Le modèle actuel autorise déjà des moteurs consommés en services pour Rust. Cette possibilité doit être évaluée avec les mêmes critères dans les trois scénarios.

Besoin	Java	Rust	.NET
Métier stock / commandes	OFBiz ou Mantle à qualifier, ou modules propres	Mêmes services métier possibles ; contrats et exploitation à mesurer	Mêmes services métier possibles ; adaptation à mesurer
Règles / processus	Flowable, KIE/Drools	Temporal Rust ou moteurs partagés ; notation métier à couvrir	Elsa, RulesEngine ou DMN partagé
Configuration contextuelle	Superposition consommable en service	Superposition, sans exclusivité liée au langage	Superposition consommable en service
Messages et transactions	Spring Modulith / Camel selon besoin	Mécanismes/services à choisir sur garanties requises	Wolverine durable inbox/outbox
Optimisation	OR-Tools, Timefold à qualifier	Solveur partagé ou interface validée	OR-Tools officiel ; service Timefold possible

Java reste une référence pertinente par ses moteurs métier et son intégration à Spring. Une proximité avec .NET sur des fonctions communes est cependant crédible. Rust peut coûter plus cher en assemblage, mais Superposition et le SDK Temporal Rust GA changent certains choix possibles. Le coût dépend du produit assemblé, des garanties et de l'équipe ; pas d'une prime uniforme par langage. [P04] [P07] [P11] [P13] [P14] [P15]

Licences et éditions : créditer seulement le périmètre effectivement réutilisable

Flowable 7 a retiré les anciennes UI OSS ; Elsa n'est pas automatiquement une suite BPMN/CMMN équivalente. ABP distingue framework libre et modules PRO. MassTransit v9 est commercial/source-available ; Camunda 8.6+ requiert une licence de production pour ses runtimes concernés. Ces produits ne sont pas tous retenus dans le classeur : il s'agit d'un contrôle des candidats, pas de charges à ajouter d'office. [P02] [P16] [P18] [P33]

Akeneo CE ne comprend pas les Reference Entities Enterprise. La gratuité de Pimcore en production sous POCL dépend notamment d'un seuil de chiffre d'affaires consolidé de 5 M€ ; elle ne doit pas être présumée pour le groupe. Medusa distingue le noyau et des fonctions Enterprise. Une édition commerciale choisie aurait une ligne de licence et de support dans le futur TCO ; elle ne serait pas masquée dans des JH de développement. [RP02] [RP03] [SO14]

5. Ce que disent réellement les références

Analystes : utiles pour comparer le périmètre

Gartner propose de choisir un DOM selon les attentes de promesse et de fulfilment ; seul son résumé public a été consulté. Forrester décrit une évolution par modules des OMS. La réimpression intégrale de sa Wave DPA 2025 éclaire les outils d'automatisation et leur gouvernance. Aucun de ces documents ne chiffre la construction de FLOW. [R01] [R02] [P22] [P32]

Une référence chiffrée à classer correctement

Le TEI KIBO de Forrester Consulting (octobre 2025, commandité par KIBO) modélise 12 personnes IT/métier à 50 % pendant six mois, soit 36 personnes-mois, et 465 000 USD internes + 300 000 USD de prestations. Le total de 765 000 USD devient 879 750 USD après majoration de risque. C'est une implémentation d'un OMS déjà développé, incluant planification et métiers ; ce n'est ni un benchmark de build Core, ni une équivalence fonctionnelle à tout FLOW. [R03]

Retours ESN : des comparables à utiliser sans extrapolation

Référence	Travail effectivement décrit	Usage dans cet audit
Thoughtworks / retail	Accélérateur IA de modernisation et extraction de spécifications	Prouve un cas de réemploi/IA ; ne chiffre pas un Core neuf
EPAM / Lowe's Canada	Assemblage D365, Sitecore, Azure et adaptations de fulfilment	Illustre les frontières produit/intégration ; JH non publiés
Capgemini / automobile	Développement et évolution d'un système de commandes	Périmètre métier comparable partiel ; équipe/effort non publiés
Flowable / Great Dane	Automatisation du suivi fournisseur autour de JD Edwards	Application sur plateforme existante ; délai non convertible en JH
DXC / ISBSG	Méthode de productivité selon taille et type de travail	Base méthodologique pour une calibration indépendante

[R07] [R08] [R09] [P19] [R05]

Aucun délai commercial de « cinq jours », « trois mois » ou « douze mois » n'a été transformé en charge. Il faudrait connaître l'équipe, son occupation, les réemplois antérieurs, le périmètre livré et les critères de qualité.

6. IA et complexité : ce qui reste à mesurer

Le gain IA est déjà matériel dans le classeur

Le scénario Java retire 3 327 JH nets, soit 19,76 % du reste après OSS et 2,495 M€. Sans ce gain, le build Java passerait de 10,135 à 12,630 M€. Il s'agit d'une sensibilité arithmétique de l'hypothèse actuelle, pas d'une recommandation de supprimer l'IA.

L'étude randomisée Microsoft/Accenture/autre entreprise observe +26,08 % de tâches terminées sur 4 867 développeurs ; elle mesure du débit avec un assistant de complétion. METR signale en février 2026 que ses nouvelles observations ne permettent pas d'isoler une ampleur fiable du gain. Le rapport ISBSG 2026 ne dispose pas d'un indicateur explicite d'usage IA dans ses données. Ces résultats justifient une expérimentation locale, pas le coefficient exact du classeur. [R10] [R11] [R04]

Préserver la logique de calcul : réemploi puis IA sur le travail restant, avec revue et correction. Une fonction supprimée par réemploi ne produit pas un second gain IA. Les gains doivent être mesurés jusqu'au résultat accepté, incluant tests génériques, corrections et documentation nécessaires.

La difficulté algorithmique doit devenir une exigence mesurable

Pour les règles, la recherche étudie notamment recouvrements et lacunes de tables DMN ; des analyseurs existent. Pour les workflows, la vérification formelle générale peut être très difficile sans que l'exécution d'un processus usuel le soit. La décision est de borner les modèles autorisés et de réutiliser les moteurs, pas de fabriquer un vérificateur universel. [P06] [P23] [P24]

Pour l'optimisation, distinguer taille des instances, contraintes dures/souples, qualité attendue de la solution et temps de calcul. Un solveur réutilisable réduit le travail d'algorithme ; il ne spécifie pas les objectifs Supply, les pénalités, l'explication et l'application des décisions.

Dimension de difficulté	Mesure proposée
Stock / réservations	Cardinalités, contention sur un article, invariants, reprises et délais de convergence
Plans / promesse / transport	Taille de problème, horizons, contraintes, temps limite et qualité acceptable
Configuration	Nombre d'axes, règles, profondeur de surcharge et effets d'une publication
Produit	Latence et débit, disponibilité, récupération, versions supportées, droits et traçabilité

Les tailles T-shirt peuvent synthétiser le reste à faire après réemploi. Elles doivent être accompagnées de ces dimensions ; elles ne constituent pas un abaque de jours.

7. Ce que je ferais évoluer ensuite

Étape 1 - Fixer les frontières. Valider les 14 fonctions et leurs sous-ensembles selon la cartographie r87. Affecter une seule fois moteurs, publication, audit, retries, optimisation et tests transverses. Clarifier transport et finance avant tout nouveau total.

Étape 2 - Comparer deux architectures de réemploi. A : modules métier développés sur frameworks. B : socle métier existant adapté au Core générique. Pour chacune, comparer Java, Rust et .NET à fonctions et qualité égales. Ne pas appeler « Make » un périmètre différent du Buy.

Étape 3 - Décrire les écarts aux composants. Pour chaque unité : couvert en standard, configuration générique, extension générique, nouveau développement ou hors périmètre. Préciser version, licence, API, défaut fonctionnel et preuve de couverture.

Étape 4 - Mesurer quatre parcours représentatifs. Stock/réservation/correction concurrente ; commande/retour/consignation avec fractionnement ; configuration versionnée avec explication d'une décision ; promesse/planification/transport avec contraintes. Le résultat accepté et les données synthétiques sont identiques entre options.

Étape 5 - Recalculer les charges résiduelles. Dimensionner les unités restantes, les multiplier par une productivité calibrée sur des réalisations comparables, puis ajouter les travaux non fonctionnels explicitement bornés. Confronter avec un chiffrage indépendant d'ESN indiquant JH par rôle et hypothèses. La consultation et ses coûts ne sont pas intégrés d'office au Core.

Étape 6 - Actualiser Comparaison et TCO après ce travail. Faire remonter uniquement les charges Core réévaluées et les éventuelles licences choisies. Conserver séparés Intégration business, déploiement, exploitation et pilotage global. Aucun recalcul n'a été effectué dans cet audit.

Une justification compacte pour le lecteur

Synthèse de 14 lignes	Fiche de preuve de 3 à 5 unités par fonction
JH Java et écarts .NET/Rust ; taille ; raison principale ; qualité de preuve	Quantité × charge unitaire ; composant qui remplace le travail ; adaptation restante ; gain IA net ; référence ou mesure ; intervalle et condition

Le gain d'un composant doit être présenté comme un **travail évité identifié moins son adaptation**. Si plusieurs composants sont complémentaires, afficher un gain de groupe ; une ventilation arbitraire en « gain par bibliothèque » donne une précision trompeuse.

Décision proposée : reprendre en premier F11, F03 et F05 ; clarifier simultanément F07/F08 ; reconstruire ensuite les différences de stacks sur F06/F11/F12/F13. Conserver le chiffrage actuel comme hypothèse de cadrage tant que cette calibration n'est pas faite. Ne pas annoncer aujourd'hui une réduction globale prétendument démontrée par le marché.

8. Sources vérifiées

Références consultées le 21 octobre 2026. Documentation évolutive : versions et licences à revalider au choix final. Les identifiants correspondent aux liens des fiches.

R05 - DXC / ISBSG - Development and Enhancement Project Productivity, Luigi Buglione

27/06/2025 | Méthode ESN / benchmark

Distinguer nature du projet, fonctions, exigences non fonctionnelles et activités d’organisation. Un taux universel de productivité ne convient pas.

R06 - SEI - Software Cost Estimation Explained

17/06/2024 | Méthode d’estimation

Relier taille, facteurs de coût et relation calibrée ; la description fonctionnelle seule ne chiffre pas l’effort.

RP01 - Akeneo PIM Community Edition

Documentation courante / date non précisée | documentation produit primaire

Édition communautaire gratuite pour structurer et enrichir les informations produits.

RP02 - Reference entities - API concepts

Documentation courante / date non précisée | documentation API primaire

Les Reference Entities possèdent attributs et cycle propre ; elles sont réservées à Enterprise Edition.

RP03 - Pimcore Open Core et licence 2026.x

Documentation courante / date non précisée | produit et licence primaires

Le modèle actuel est POCL, source disponible. Gratuité de production conditionnée notamment par un chiffre d'affaires groupe au plus égal à 5 M€, avec exceptions prévues par la licence.

SO01 - Apache OFBiz - Business Users

Page non datée ; consultée 2026-10-21 | Documentation produit primaire OSS

Stocks multi-entrepôts, lots/séries, expédition/réception/retours, catalogue variantes, commandes, achats et fabrication intégrés.

SO03 - Moqui - Mantle Business Artifacts

Page courante non datée, consultée 2026-10-21 | Documentation produit primaire OSS

UDM/USL, achats, ventes, inventaire, réservations, projections, expédition et retours clients/fournisseurs. La page expose entités/services/écrans métier.

P01 - Flowable Open Source

Documentation courante / date non précisée | Documentation / code primaire

Moteurs Java BPMN, CMMN et DMN sous Apache 2.0 ; usage embarqué ou par API. La plateforme commerciale ajoute des capacités qui ne se déduisent pas de la seule présence des moteurs.

P06 - Apache KIE Drools DMN

Documentation courante / date non précisée | Documentation / code primaire

Runtime DMN Java et analyse statique des tables : lacunes et recouvrements. Page ouverte une première fois ; réouverture ultérieure en erreur, sans modification de la conclusion.

P17 - Microsoft RulesEngine

Documentation courante / date non précisée | Documentation / code primaire

Moteur .NET sous MIT pour règles décrites en JSON et expressions. Il ne démontre pas à lui seul une gouvernance complète de configuration ; ne pas lui attribuer les capacités d'un produit de gestion de règles absent.

P27 - Microsoft Dynamics 365 Commerce - DOM rules

Documentation courante / date non précisée | Documentation / code primaire

Paramètres de dates, activation et contraintes ; protections de stock par produit/variante/lieu, priorités et limites de fractionnement. Montre une couverture produit générique à comparer famille par famille.

P28 - SAP S/4HANA Fashion - Explaining Supply Assignment

Documentation courante / date non précisée | Documentation / code primaire

Distingue affectation, contrôles de libération et désaffectation ; règles de release déterminées selon organisation commerciale, canal, division, type de commande et client. Paramétrage multidimensionnel concret pour challenger F02/F11.

SO05 - Mantle - AssetServices.xml

Master consulté 2026-10-21 | Implémentation primaire OSS

Services de disponibilité par produit/facility/propriétaire/pool, recalcul des agrégats depuis détails, réservations, réaffectation et contrôle de validité.

SO06 - ERPNext - Immutable Ledger

Page non versionnée ; consultée 2026-10-21 | Documentation technique primaire OSS

Conservation des écritures originales, contre-passation, correction et reposting contrôlé ; les changements antidatés peuvent nécessiter recalcul des états suivants.

SO07 - ERPNext - Stock Reservation

Réservation introduite version15 ; documentation courante ; licence branche develop | Documentation et licence primaires OSS

Réservation depuis Sales Order/Pick List, réservation automatique après achat/réception et dé-réservation. ERPNext GPLv3.

SO08 - Odoo 19 stock - manifest et stock_quant.py

Branche19.0 consultée 2026-10-21 | Code/licence primaire OSS

Le module communautaire stock déclare LGPL-3 ; modèle quantités/propriétaire/lot/company/emplacement, réservations et différence d'inventaire.

SO17 - Microsoft - Inventory Visibility reservations

Support sales orders10.0.33+ ; documentation actuelle | Documentation technique commerciale primaire

Réservation souple contrôlant disponibilité et renvoyant identifiant, consommations/offsets et annulation ; sémantique dimensions configurable.

SO18 - Microsoft - Order promising

Mise à jour2026-04-21 | Documentation algorithmique commerciale primaire

ATP cumulatif avec look-ahead, horizon et réceptions/besoins ; distinction CTP qui ajoute la capacité. Différences de fonctions entre moteurs de planning.

SO20 - IBM Sterling Intelligent Promising - optimization objectives

Documentation courante consultée 2026-10-21 | Documentation technique commerciale primaire

Consultation : contenu primaire indexé ; ouverture HTML vide ou refusée, sans lecture directe complète.

Critères déterministes coût, capacité, traitement et vitesse ; fonctions prédictives premium stockout/markdown. Arbitrage nombre d'expéditions/coût/capacité.

SO04 - Mantle USL - OrderToCashBasicFlow

Branche master à la consultation 2026-10-21 | Code et test primaire OSS

Test Spock créant une commande et vérifiant réservations/données du cycle. Licence CC0 1.0 avec grant de brevet.

SO12 - Medusa Order Module

v2.22 affichée | Documentation technique primaire OSS

Ordres, drafts, modifications, retours, échanges et réclamations ; changements versionnés.

SO14 - Medusa - licences principale et Enterprise

Develop à la consultation 2026-10-21 | Licences primaires

MIT sauf fonctionnalités Enterprise ; RBAC/SSO explicitement couverts par licence Enterprise.

P02 - Flowable Open Source 7.0.0 Release

2023-09 | Documentation / code primaire

La version 7 retire les anciennes UI et les moteurs Form/Content. Le nouveau Design est accessible gratuitement dans le cloud pour exporter BPMN/CMMN/DMN ; cela ne rend pas toute la suite Work disponible en OSS.

P04 - Elsa 3.8 API & Client

Documentation courante / date non précisée | Documentation / code primaire

API de création, publication, import/export et interrogation de définitions versionnées ; diagnostic des instances et permissions d'accès. La couverture ne prouve pas une équivalence CMMN ou une gouvernance de toute configuration Supply.

P05 - Elsa Workflows source repository

Documentation courante / date non précisée | Documentation / code primaire

Projet de workflow .NET sous MIT ; code et applications du projet accessibles. Évaluer la couverture de chaque artefact retenu.

P11 - Temporal Rust SDK generally available

2026-09-23 | Documentation / code primaire

SDK Rust annoncé GA en septembre 2026 : workflows/activités/workers, replay d'historique, intégration Tokio. Les activités à effets doivent rester idempotentes ; workflows déterministes. Pas de preuve d'une suite BPMN/CMMN avec portail de tâches équivalente.

P19 - Great Dane streamlines order lifecycle management with Flowable

Documentation courante / date non précisée | Cas client publié par le fournisseur

Supplier LEAN Alert autour de JD Edwards : étude du périmètre en décembre, production début mars selon le client. Application de suivi de problèmes fournisseurs sur une plateforme existante.

SO13 - Medusa Fulfillment Module

v2.22 affichée | Documentation technique primaire OSS

Prestations fulfillment, fournisseurs tiers, lieux/règles et méthodes de livraison/retrait.

SO24 - OneStock - Monsoon inventory unification

Projet démarré en2017 selon texte ; page courante non datée | Cas client primaire éditeur

Ship from Store livré en moins de5 mois ; raccordement ERP Aptos Mercatus et DHL.

SO26 - Fluent Commerce / Pivotree - Psycho Bunny

Mise en production fin octobre2022 ; publication PDF non datée | Cas client primaire éditeur et intégrateur

MVP Home Delivery/Global Inventory/Order Orchestration mis en œuvre en14 semaines ; Click & Collect prévu dans la phase suivante.

SO29 - Google OR-Tools - Vehicle Routing

Mise à jour2025-01-22 | Documentation algorithmique primaire OSS

Routage avec capacités, fenêtres temporelles et ressources ; limites d'optimalité et explosion combinatoire expliquées. Moteur disponible gratuitement.

RP10 - Gotenberg - Docker API for document conversion

Documentation courante / date non précisée | documentation et code primaires

Service de conversion HTML/Office en PDF utilisant notamment Chromium et LibreOffice, avec opérations PDF. Dépôt sous MIT.

RP12 - Apache FreeMarker

Documentation courante / date non précisée | documentation primaire

Moteur de templates Java pour produire du texte à partir de données ; projet Apache.

RP13 - Scriban

Documentation courante / date non précisée | code et documentation primaires

Moteur de templates et de script .NET, avec compatibilité Liquid ; licence BSD-2-Clause.

RP14 - Tera

Documentation courante / date non précisée | code et documentation primaires

Moteur de templates Rust inspiré de Jinja2/Django ; licence MIT.

RP16 - Introduction to OR-Tools

2024-08-28 | documentation primaire

Bibliothèque de résolution couvrant plusieurs familles d'optimisation ; licence Apache 2.0.

RP17 - Install OR-Tools

2026-03-18 | documentation primaire

Moteur C++ avec distributions/interfaces officielles Python, Java et C#. Rust n'est pas dans cette liste officielle.

RP18 - Timefold Solver

Documentation courante / date non précisée | documentation primaire

Solveur Java/Kotlin, Community Apache 2.0, issu d'OptaPlanner, avec intégration à l'écosystème Java.

RP19 - Timefold Solver commercial editions

Documentation courante / date non précisée | documentation et licence primaires

La documentation distingue l'édition commerciale de la Community ; analyse des scores, recommandations, multithreading et autres capacités avancées figurent dans l'offre commerciale.

RP20 - frePPLe current README and license

Documentation courante / date non précisée | documentation et licence primaires

Outil de planification et prévision orienté fabrication, avec Community MIT et éditions commerciales. La documentation actuelle contredit les brochures historiques décrivant une Community AGPL.

RP22 - Takko Fashion to Use RELEX Solutions to Optimize its Supply Chain

2024-11-25 | cas client éditeur primaire

Décrit un projet Fashion couvrant prévisions, allocation, réassort, redistribution, tailles/couleurs et markdown, sur une plateforme intégrée.

RP25 - Minimum Cost Flows

Documentation courante / date non précisée | documentation algorithmique primaire

Modélise des flux sous capacités, demandes et coûts linéaires ; exemples Java et C# disponibles avec le même solveur.

RP26 - Two-level lot-sizing with inventory bounds

2016-04-08 | article scientifique primaire

Consultation : résumé scientifique primaire ; article intégral non audité.

Le placement des bornes de stock et l'interdiction de fractionner une demande changent la complexité : certaines variantes admettent un algorithme polynomial, d'autres sont NP-difficiles.

RP27 - Inventory Management of a Fast-Fashion Retail Network

2009-08-12 | article scientifique et cas Zara primaire

Consultation : résumé scientifique primaire ; article intégral non audité.

L'allocation Zara combine prévisions, stock par taille, politique de retrait d'exposition et programmation mixte en nombres entiers. Le cas montre qu'un modèle Fashion comprend plus que l'appel d'un solveur.

SO15 - SAP S/4HANA - Supply Assignment

Version2025 FPS01 février2026 indiquée par source indexée | Documentation produit commerciale primaire

Consultation : contenu primaire indexé ; ouverture HTML vide ou refusée, sans lecture directe complète.

Affectation en pénurie aux commandes/contrats/STO, contrôles de libération, modes immédiat/batch/API ; fonction SUPPLY_ASSIGNMENT_01.

SO27 - Intelligent Warehouse Allocator for Optimal Regional Utilization

Soumis2020-07-09, Girish Sathyanarayana et Arun Patro | Recherche primaire appliquée Fashion

Consultation : résumé scientifique primaire ; article intégral non audité.

Myntra : affectation d'inventaire acheté à des entrepôts avec capacités, demande régionale et minimisation des redistributions ; programmation entière et prévision.

SO28 - A Deterministic Model for the Transshipment Problem of a Fast Fashion Retailer under Capacity Constraints

Soumis2019-12-27 ; Naderi, Kilic, Dasci | Recherche primaire Fashion

Consultation : résumé scientifique primaire ; article intégral non audité.

Réseau LC Waikiki450+ magasins : modèle linéaire entier mixte et heuristique ; qualité des solutions dépend du modèle/stratégie, pas seulement de la présence d'un solveur.

P07 - Juspay Superposition source repository

Documentation courante / date non précisée | Documentation / code primaire

Service de configuration contextuelle écrit en Rust sous Apache 2.0, avec héritage/surcharge, UI, API, audit et versions. Candidat de réemploi ; aucune mesure de charge transférable.

P08 - Superposition validation functions

Documentation courante / date non précisée | Documentation / code primaire

Validation de valeurs et de combinaisons contexte/surcharges ; fonctions testables avec brouillon/publication, exécutées dans un contexte Node vm limité en durée. Ce dispositif concerne les fonctions décrites, sans preuve d'un workflow complet d'approbation de chaque paramètre.

P09 - Superposition workspaces

Documentation courante / date non précisée | Documentation / code primaire

Schéma PostgreSQL isolé par workspace ; audit, version de configuration et options de validation. Paramètre d'auto-approbation des expérimentations ; aucune généralisation automatique aux publications coordonnées de FLOW.

P10 - Enhanced configuration management & reliability in Hyperswitch

2026-06-12 (édition indiquée) | Documentation / code primaire

L'équipe Hyperswitch annonce l'adoption de Superposition pour sa configuration, avec cache/polling, validation, ciblage et rollback. Le document distingue des migrations et une intégration console encore prévues.

P25 - Decision Provenance: Harnessing data flow for accountable systems

2018-04-16 ; version consultée 2019-11-15 | Publication / standard primaire

La provenance décisionnelle relie entrées, décisions et effets dans des systèmes interconnectés. Ce périmètre est plus riche qu'un historique des changements de paramètres.

P26 - W3C PROV-DM: The PROV Data Model

2013-04-30 | Publication / standard primaire

Modèle standard de provenance reliant entités, activités, agents et dérivations ; vocabulaire réutilisable pour HS-023.

P13 - Spring Modulith - Working with Application Events

Documentation courante / date non précisée | Documentation / code primaire

Registre de publication écrit avec la transaction métier, suivi d'achèvement et resoumission ; gestion du cycle de publication enrichie depuis 2.0. Candidat pour un produit modulaire sans imposer une architecture de microservices.

P14 - Wolverine - Durable Inbox and Outbox Messaging

Documentation courante / date non précisée | Documentation / code primaire

Mécanismes inbox/outbox persistants et intégration transactionnelle ; modes durables à activer et configurer. Site du projet indique MIT.

P18 - Massient - MassTransit licensing FAQ

Documentation courante / date non précisée | Documentation / code primaire

La FAQ indique que MassTransit v9 adopte une licence commerciale/source-available ; les anciennes versions permissives ne constituent pas la même hypothèse. Wolverine reste une option MIT documentée.

P15 - ABP framework source repository

Documentation courante / date non précisée | Documentation / code primaire

Socle ASP.NET Core avec architecture modulaire, infrastructure transverse, modules et modèles de départ. Licence du dépôt LGPL-3.0.

P16 - ABP - Free license versus commercial PRO

Documentation courante / date non précisée | Documentation / code primaire

Distinction explicite entre framework/modules de base gratuits et modules/interfaces/générateur PRO. Tenant Management de base n'est pas tout le module SaaS commercial ; certaines interfaces de gestion avancée sont payantes.

P30 - Testcontainers OSS overview

2024 (support Spring I/O) | Documentation / code primaire

Dépendances réelles jetables pour tests : bases, brokers et navigateurs ; Java, .NET et Rust figurent parmi les langages supportés.

P31 - OWASP Application Security Verification Standard 5.0.0

2025-05-30 | Documentation / code primaire

Référentiel de contrôles de sécurité pour conception, développement et vérification ; exigences identifiables et exportables pour une matrice de qualification.

P33 - Camunda 8 Licensing

Documentation courante / date non précisée | Documentation / code primaire

Pour 8.6+, les composants runtime cités nécessitent une licence Enterprise en production ; Desktop Modeler reste MIT et certains composants connecteurs sont Apache 2.0.

R01 - Gartner - How to Select a Distributed Order Management System to Match Your Vision

18/12/2024 | Résumé public

Choix DOM selon la promesse et le fulfilment. Recherche intégrale non consultée ; aucun abaque de JH accessible.

R02 - Forrester - OMS and a Story of Augmented Evolution

21/01/2025 | Analyse publique

Évolution modulaire du marché OMS ; preuve de pertinence du réemploi par composants, pas de coût de fabrication.

P22 - The Forrester Wave - Digital Process Automation Software, Q3 2025

2025-08-20 | Rapport analyste intégral consulté

Les références interrogées décrivent surtout des équipes professionnelles. Une partie de la gouvernance est assurée par l'intégration aux outils IT de gestion des sources et déploiements. Cela incite à examiner le réemploi de la chaîne existante.

P32 - Gartner Magic Quadrant for Business Orchestration and Automation Technologies

2025-10-15 | Résumé public d’analyste

Catégorie de plateformes réunissant orchestration, connectivité et capacités agentiques ; 20 fournisseurs dont SAP, Microsoft, Flowable et Camunda.

R03 - Forrester Consulting - Total Economic Impact of KIBO Order Management

10/2025 | Étude commanditée par KIBO

Cas composite issu de six décideurs de quatre organisations. Chiffres de planification et implémentation ; aucun coût de développement du produit générique.

R07 - Thoughtworks - Major multinational retailer

Date non précisée | Cas ESN, éditeur du cas

Accélérateur de modernisation et extraction de spécifications avec IA. Les réductions annoncées concernent cette activité ; aucune transposition au build complet.

R08 - EPAM - Unified customer-centric shopping experience for Lowe’s Canada

Mise en service 08/2019 | Cas ESN

Assemblage Dynamics 365, Sitecore et Azure avec adaptations de stocks et fulfilment. Pas d’effort ventilé publié ; cas d’intégration client.

R09 - Capgemini - Car Order Management System

Date non précisée | Cas ESN

Système de commandes automobile sur mesure. Pas de budget, de JH ni de disponibilité de l’équipe publiés.

R10 - Microsoft Research - Three Field Experiments with Software Developers

06/2025 | Essais randomisés

4 867 développeurs, +26,08 % de tâches terminées en moyenne. Une hausse du débit ne se transpose pas directement en baisse du coût complet.

R11 - METR - Changing our Developer Productivity Experiment Design

24/02/2026 | Recherche expérimentale

Le signal récent d’accélération est jugé peu fiable pour en mesurer l’ampleur, en raison des sélections et de la mesure du travail avec agents multiples.

R04 - ISBSG - Impact of AI-Assisted Development on Software Productivity and Delivery Speed

02/2026 | Étude observationnelle

Pas de champ explicite mesurant l’usage IA dans le référentiel. Comparaisons temporelles et variables indirectes ; causalité non établie, aucun coefficient FLOW.

P23 - Semantics, Analysis and Simplification of DMN Decision Tables

2018-11 | Publication / standard primaire

Article des auteurs : analyse géométrique des recouvrements/lacunes ; contrôle naïf des paires en O(d × R²), dépendance forte au nombre d'axes pour des analyses plus générales. Une heuristique de simplification évite de chercher systématiquement l'optimum général.

P24 - The complexity of soundness in workflow nets

2022-01-14 | Publication / standard primaire

Résultats EXPSPACE-complets pour les variantes classique/structurelle, PSPACE-complet pour la généralisation étudiée. Ils concernent la vérification formelle de réseaux de workflow.

9. Traçabilité et intégrité de l’audit

Classeur source : FLOW_Comparaison_TCO_trois_stacks.xlsx. Onglet FLOW Core System, synthèse et 14 fonctions ; Charges Core et Stacks Core pour la décomposition OSS/IA ; Comparaison pour les coûts de build. Valeurs mémorisées du classeur et modèles de calcul locaux rapprochés.

SHA-256 du classeur au début et à la fin :
64c1702b1f372dbf973e6f6e34daf51eb07bbd9e461683657d2f4a478996b4a9

Cartographie source : C:/Dev/Beaumanoir Cartographie/modeles/release/2026-10-21.3/model.yaml, publication r87. Lecture ciblée des domaines Supply et des hotspots associés ; aucun changement de ce modèle.

SHA-256 du modèle publié :
6436a86f95655d40cffb508ea6a63d4456f7a4487fb1aa5dd5862ac3fb7dc101

Travail réalisé : inspection du chiffrage et de son origine ; rapprochement du périmètre ; recherches de documentation, dépôts, licences, publications algorithmiques, analystes et retours ESN ; analyse des possibilités de réemploi et de leurs limites.

Limites : aucun prototype de performance ni test d’adaptation produit n’a été exécuté ; aucun fournisseur n’a été sollicité ; les recherches intégrales Gartner sous accès réservé n’ont pas été consultées. Un dépôt ou une documentation prouve une capacité déclarée, pas son adéquation complète à FLOW ni sa charge d’industrialisation.

Résultat : audit et propositions seulement. Classeur, formules, Comparaison, TCO et cartographie inchangés. Ce PDF est un livrable séparé.